

Recordatorio

En la sesión anterior, se te fue asignado directamente por Don Donnie, dueño del Casino Black Cat, la tarea de generar un prototipo del clásico juego de la ruleta. En esa primera sesión, trabajaste directamente en una interfaz tipo CLI (Command Line Interface) a propósito, ya que este fue un primer paso para identificar qué opciones y funcionalidades básicas debería tener el juego, sin poner foco todavía en cómo se ve.



Figura 1: Don Donnie

Contexto

Para evitar problemas de seguridad, propios de un sistema sin mecanismos de autenticación, causados por un hacker con malas intenciones, se te ha solicitado generar un registro/inicio de sesión para usuarios, con tal proteger la sesión y los datos de cada usuario.

En esta sesión no se trabajará con manejo de archivos, por lo que las credenciales estarán «hardcoded» en arreglos dentro del programa. En futuras iteraciones serán incorporadas las funciones lectura y escritura de archivos para almacenar usuarios de manera persistente.

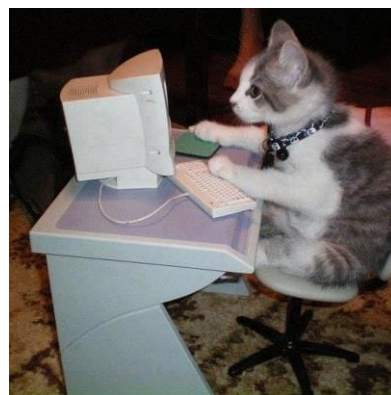


Figura 2: Hacker en cuestión

Objetivos

Como objetivo principal de esta sesión, se espera generar un sistema basado en ventanas que permita al usuario iniciar sesión, la cual responda de la siguiente manera al presionar el botón de «Ingresar»:

- Si las credenciales son correctas, debe aparecer un mensaje de éxito (JOptionPane) y abrirse la ventana generada en la [«Guía de Trabajo Autónomo»](#).
- Si son incorrectas, debe aparecer un mensaje de error (JOptionPane).

Requisitos

- Cada función debe encargarse de una única cosa (evitar el «método dios»).

- Cada función debe tener entre 5 a 10 líneas.
- Commits claros, uno por funcionalidad.
- El método `main` solo llama al menú (no contiene lógica).
- Uso de `final` para constantes (evitar números mágicos).
- Los nombres de las funciones deben tener sentido y ser claros.

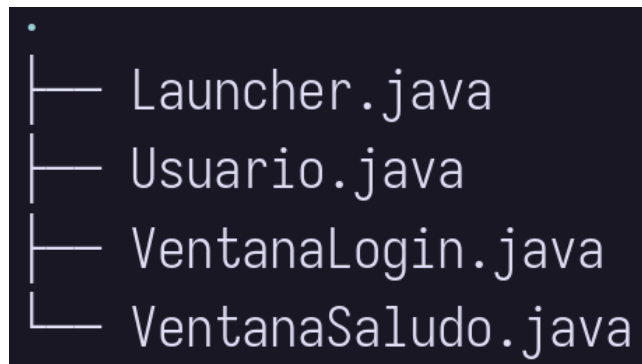


Figura 3: Estructura de referencia del proyecto

Clase Usuario

Nota

La siguiente clase será entregada como ejemplo y referencia, ya que los términos de clases y objetos serán estudiados en sesiones posteriores a esta. Por ahora, el enfoque de la sesión estará en manejo de ventanas, razón por la cual se entrega el código de la clase «Usuario».

```
public class Usuario {  
    private String username;  
    private String password;  
    private String nombre;  
  
    public Usuario(String username, String password, String nombre) {  
        this.username = username;  
        this.password = password;  
        this.nombre = nombre;  
    }  
}
```

```

// Verifica si las credenciales coinciden
public boolean validarCredenciales(String u, String p) {
    return this.username.equals(u) && this.password.equals(p);
}

public String getNombre() {
    return nombre;
}
}

```

Clase VentanaLogin

```

import javax.swing.*;

/**
 * Clase que representa la ventana de Login del Casino Black Cat.
 * Permite ingresar credenciales para acceder al sistema.
 */
public class VentanaLogin {
    // --- Lista dinámica de usuarios ---
    public static final List<Usuario> USUARIOS = new ArrayList<>();

    // --- UI ---
    private final JFrame frame = new JFrame("Login - Casino Black Cat");
    private final JLabel lblUsuario = new JLabel("Usuario:");
    private final JTextField txtUsuario = new JTextField();
    private final JLabel lblClave = new JLabel("Clave:");
    private final JPasswordField txtClave = new JPasswordField();
    private final JButton btnIngresar = new JButton("Ingresar");

    /**
     * Constructor que inicializa la ventana de login.
     * Configura el tamaño, los componentes y los eventos.
     */
    public VentanaLogin() {
        // TODO: inicializar usuarios hardcodeados
        // TODO: inicializar y configurar la ventana
    }
}

```

```

/**
 * Muestra la ventana en pantalla.
 * Debe centrarla y hacerla visible.
 */
public void mostrarVentana() {
    // TODO: mostrar ventana
}

/**
 * Maneja el evento de login al presionar el botón.
 * Debe validar credenciales y abrir la siguiente ventana o mostrar error.
 */
private void login() {
    // TODO: implementar lógica de login
}

/**
 * Valida las credenciales del usuario contra el arreglo hardcoded.
 *
 * @param u nombre de usuario ingresado
 * @param p clave ingresada
 * @return el nombre del usuario si es válido, o cadena vacía si no coincide
 */
private String validarCredenciales(String u, String p) {
    // TODO: recorrer arreglo y validar credenciales
    return "";
}

/**
 * Abre la ventana de registro para crear un nuevo usuario.
 * Debe cerrar la ventana actual e invocar a VentanaRegistro.
 */
| void abrirRegistro() {
    // TODO: abrir ventana de registro y cerrar login
}
}

```

Desafío: Registro de usuario

Como extensión a esta sesión de inicio de sesión, se propone la implementación de una ventana de registro de usuario, con el objetivo de permitir que un nuevo jugador del Casino Black Cat pueda crear su propia cuenta, ingresando su usuario, clave y nombre completo. Para esto se debe considerar:

1. Validar que todos los campos estén completos antes de aceptar los datos.
2. En caso de éxito mostrar un mensaje confirmando que el usuario fue registrado. Para esto se debe agregar la nueva cuenta a la estructura de datos utilizada para el login (lista de usuarios).
3. Finalmente, la aplicación debe volver a mostrar la ventana de login para permitir el acceso con las nuevas credenciales.

Referencias

- [1] [Documentación] Apuntes Java - Jesús. <https://jessustm.github.io/ICC490-1-P00/>
- [2] [Documentación] Apuntes List - Jesús. <https://jessustm.github.io/ICC490-1-P00/1.Apuntes/11.List/>
- [3] [Tutorial] Vincular un proyecto Java en IntelliJ IDEA con GitHub. <https://github.com/JesusTM/ICC490-1-P00/blob/main/2.Reportes/%5BRT-P00-01%5D%20GitHub%20-%20IntelliJIDEA.pdf>
- [4] [Tutorial] Cómo usar Git y GitHub. <https://github.com/JesusTM/ICC490-1-P00/blob/main/2.Reportes/%5BRT-P00-02%5D%20Uso%20de%20GitHub.pdf>
- [5] [Tutorial] Cómo crear un menú en Java (ciclo do-while). <https://tutospoo.jimdofree.com/tutoriales-java/estructuras-repetitivas/ciclo-do-while-1-men%C3%BA/>